Activity- R and SQL

Kenneth Heisler

R and SQL using the RSQLite package. (12 questions, 39 pts. total)

In this lab activity, you will use the R package RSQLite to gather data from a relational database by executing queries in SQL, the language of relational databases. Once data has been retrieved, you will explore the ggplot2 and other libraries to visualize the results.

First, you will connect to the ACME relational database. This database is a file, ACME.db, created by the SQLite program. Then you will run a few simple queries and explore the results.

A query returns a result set, which is represented by a dataframe in R, upon which you can perform any dataframe operations.

Your skills as an SQL query writer will be useful in this activity.

Part 1: Run SQL queries and Visualize the Result Set.

This part uses the ACME database.

Connecting to the database

To connect to the database, a software module called a "driver" is created. The driver provides a connection object which is a stream that connects to the database and can be used to execute queries and return result sets.

The following statement connects to the ACME database file. The acme.db object is used to execute queries and receive result sets from that database. You will need to specify the path to the location of the ACME.db file on your local machine.

```
# specify the path to the ACME.db file- omit the last / from the end of your path. Do not include the d
# example: /users/myname/documents/info248/week13

db.path<-
acme.db <- dbConnect(SQLite(), dbname=paste(db.path, "ACME.db", sep="/"))
```

If the connection is made, the acme.db object will be created and should appear in the Environment. If you get an error, it is most likely an error in the path or file name.

Check the connection.

Once the connection has been made, you can explore the structure of the database. This statement will display a list of the tables in the ACME database. If you don't see the list of table names, the connection was not made. Make sure the file path and name are correct.

```
dbListTables(acme.db)
```

```
## [1] "Customer" "Invoice"  "Line"     "Product"  "Vendor"
```

Q1 (2pts): List the names of the tables returned from the call above in the order they appear.

Customer, Invoice, line, Product, Vendor

Execute SQL queries.

Now that you have connected to the database and viewed its tables, you are ready to write and execute some SQL queries to retrieve data.

First, you will execute a query that retrieves all of the data in from the Customer table in the ACME database.

The dbGetQuery function takes the connection object and the query string as parameters. It returns the result set as a dataframe.

```
all.customers = dbGetQuery(acme.db, 'select * from Customer' )
```

Q2 (3pts): 1- what kind of object is all.customers? 2- How many customers are returned by this query? 3- How many columns are in the Customer table?

1- data.frame-list , 2- 19 , 3- 6

One way to visualize the result set of a query (which is a dataframe in this context) is to render it as a table. The grid and gridExtra packages provide a way to create fairly nice looking tables that can be customized.

Note that you will generally present tables from fairly small result sets as smaller tables are easier to read. This example uses all of the rows of the Customer table, which is small, and so it serves as a demo for what can be done. Generally speaking, you would not display a lot of data in a tabular form.

Databse vs. Presentation Names

Notice that the column names in the Customer table in the ACME database are not always in presentable form. That's because database names, like names in programming languages, have to conform to specific rules. For example, there cannot be any white space characters in database table or column names. Furthermore, all names in a database will follow a naming convention. For example, names may be in all caps, or will use underscores, or will often be abbreviated, e.g.: FIRSTNAME, first_name, FName. The naming convention should be consistent throughout the entire database.

This works for the database but not necessarily for presentation purposes. Displaying raw database names for presentation purposes is not appropriate, and you want to demonstrate that you took care with the way your reports and visualizations look.

Q3 (2pts): create a variable called "colnames" that will be used in a table below. Assign it to a vector of these strings that are more appropriate for presentation purposes: Customer ID, First Name, Last Name, Initial, Areacode, Phone.

```
colnames<- c("Customer ID", "First Name", "Last Name", "Initial", "Areacode", "Phone")
```

Execute this call to the grid.draw function with a nested call to the tableGrob function is made to produce the table. The table can be customized by specifying themes. Here, the default theme is used with a custom font size. The size of the table font is controlled by the base_size parameter. This may take a few seconds to execute.

Note: The term "grob" is an abbreviation of "grid graphical object". https://bookdown.org/rdpeng/RProgDA/the-grid-package.html There are many R packages that can render tables. This is only one possibility.

Write a query

Usually, we want information about specific questions we have. That means we will write queries with several constraints.

Q4 (5pts): In this question, you will construct a query that returns the ProductCode, Description, and CurrentStockValue for all products that have a CurrentStockValue greater than $200. The CurrentStockValue is a computed column which is the product of UnitPrice and the QtyInStock. The results should be ordered by CurrentStockValue in descending order.

Although you can develop this query in R, it is far easier to write and perfect your query in a query editor, such as DB browser which is connected to the ACME database. Remember to start with a simple version of the query, execute it, then add another part, execute, etc. For example, do not add the constraint of >$200 until you have added the computed column to the query. Develop the query incrementally, not all at once, as it is easier to develop and troubleshoot.

Once you have the finished the query, plug it in to a call to dbGetQuery and assign the result to a variable called cur.stock.vals.

```
cur.stock.vals <- dbGetQuery(acme.db ,
          'Select ProductCode, Description, UnitPrice * QtyOnHand as CurrentStockValue
            From Product
            where CurrentStockValue > 200
            order by currentStockValue DESC;')
```

Q5 (3pts): Display the result from the query above in a table in the same format as above using the grid.draw and tableGrob functions with size 6 font (the base_size parameter). You do not need to make new column names as the names are presentable enough, so omit the "cols" parameter. Your table should have 16 rows, and the largest CurrentStockValue should be 1798.93.

```
grid.draw(tableGrob(cur.stock.vals, theme=ttheme_default(base_size = 6) ))
```

	ProductCode	Description	CurrentStockValue
1	657432	Husqvarna chain saw	1798.93
2	490930	Alpine Shed	1392.24
3	981004	Rolling Work Bench	792.00
4	756342	Acer Monitor 21.5"	639.84
5	197832	Acer Monitor 18.5"	479.84
6	987234	B\&D jigsaw	329.76
7	819190	Jawbone Prime Bluetooth Headset	319.20
8	981374	Cantilever Storage Unit	312.00
9	734644	Robot Roomba Model 440	298.50
10	734623	American Red braided rug 7x10	287.40
11	171859	700watt Microwave	272.80
12	223898	DeWalt cordless drill	272.65
13	009832	Digital photo frame	252.96
14	847721	Haier 22" 1080p 60hz LED LCD HDTV	249.96
15	423132	Claw hammer	228.85
16	837554	B\&D Power painter	219.98

Visualizing data with ggplot2

The following query retrieves aggregate counts of the number of customers per area code.

Q6 (4pts): Display the results in a table using the same format as the previous tables, however make this table's base_size 10. Also, do not display row numbers. That can be done by including the "rows" parameter set to the value NULL.

```
grid.draw(tableGrob(areacode.freqs, rows = NULL, theme=ttheme_default(base_size = 10) ))
```

Areacode	Customers
413	10
615	1
617	8

Q7 (2pts): 1- What is the areacode with the most customers? 2- How many customers have that areacode?

1- 413 , 2- 10

This table is readable, but let's look at a bar chart of the same results.

In the code below, the ggplot2 library is used to display a bar chart of area codes and the number of customers for each areacode in the database.

Note the call to reorder so that the x axis is ordered by the frequencies (of customers) in ascending order. (If you added a minus sign - before the Customers parameter in the reorder call it would sort in descending order).

You are now done with the ACME database. Always clean up by closing the connection:

Part 2: The Chinook Database.

In this part, you will write queries and display results using the Chinook database.

```
chinook.db <- dbConnect(SQLite(), dbname=paste(
```

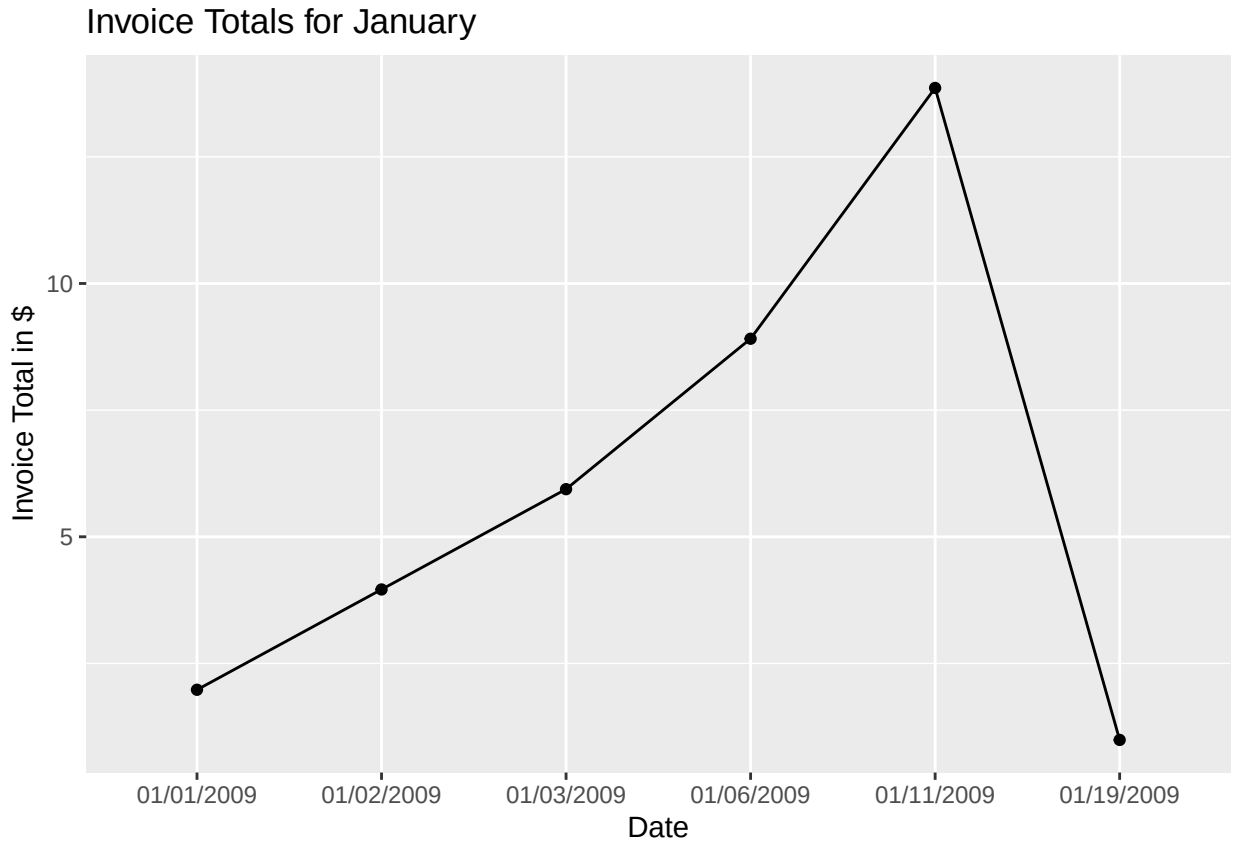
Q8 (5pts): Write and execute a query to get all columns from the invoice table for invoices for the month of January only. (invoice date >= January 1, 2009 and invoice date < February 2, 2009). Use the date format yyyy-mm-dd as that is the format the chinook database uses. Assign the result to the variable res.jan. You should have six rows in the result set.

```
res.jan<- dbGetQuery(chinook.db,
'Select *
from Invoice
where (InvoiceDate  >= "2009-01-01" and InvoiceDate <= "2009-01-31");')
```

The following code plots the totals with the amount on the vertical axis and the date on the horizontal axis.

Plot Invoice Totals

```
areacode.chart <- ggplot(data=res.jan, aes(x=format(as.POSIXct(InvoiceDate),format='%m/%d/%Y'), y=Total
areacode.chart + geom_line() + geom_point()+
   xlab("Date") + ylab("Invoice Total in $") + ggtitle("Invoice Totals for January")
```



Q9 (3pts): What are the values and dates of the highest invoice total in the month of January? The lowest? Describe any trend you observe in the data. >Highest: 13.86 - 1/11/2009 Lowest: 0.99 1/19/2009 Trend: The trend seems to increase until a vertex at 1/11/2009 where it then drops drastically.

Invoice Query

Q10 (5pts): Write and execute a query to retrieve all invoices that customers from London have generated. Select only these columns: CustomerID, First Name, Last Name, Email, InvoiceID, Invoice Date, Billing City, and Billing Country. Sort the result by alphabetical order on customer last name. Assign this call to

a variable named london.invoices. Note: the city data type is a string, which needs to be in quotes, so you will have to "escape" the quotes in the query string.

```
london.invoices<- dbGetQuery(chinook.db,
'Select Customer.CustomerID, Invoice.CustomerId, Customer.FirstName, Customer.LastName, Customer.Email,
 From Invoice, Customer
 where Invoice.BillingCity = "London"
 order by Customer.LastName asc; ')
```

Q11 (2pts): 1- How many invoices were generated from London customers? 2- How many unique London customers generated invoices? > 1- 826 , 2- 59

Q12 (3pts): Now use the result from the query above to generate a table in the same way you did in question 5 above, except set the base_size to 6.

```
grid.draw(tableGrob(london.invoices, theme=ttheme_default(base_size = 6) ))
```

03	47	53	Lucas	Mancini	lucas.mancini@yahoo.it	238	2011−11−21 00:00:00	London	United Kingdo
04	47	53	Lucas	Mancini	lucas.mancini@yahoo.it	261	2012−02−23 00:00:00	London	United Kingdo
05	47	53	Lucas	Mancini	lucas.mancini@yahoo.it	283	2012−05−27 00:00:00	London	United Kingdo
06	47	53	Lucas	Mancini	lucas.mancini@yahoo.it	335	2013−01−15 00:00:00	London	United Kingdo
07	10	52	Eduardo	Martins	eduardo@woodstock.com.br	11	2009−02−06 00:00:00	London	United Kingdo
08	10	52	Eduardo	Martins	eduardo@woodstock.com.br	140	2010−09−13 00:00:00	London	United Kingdo
09	10	52	Eduardo	Martins	eduardo@woodstock.com.br	163	2010−12−16 00:00:00	London	United Kingdo
10	10	52	Eduardo	Martins	eduardo@woodstock.com.br	185	2011−03−20 00:00:00	London	United Kingdo
11	10	52	Eduardo	Martins	eduardo@woodstock.com.br	237	2011−11−08 00:00:00	London	United Kingdo
12	10	52	Eduardo	Martins	eduardo@woodstock.com.br	358	2013−05−01 00:00:00	London	United Kingdo
13	10	52	Eduardo	Martins	eduardo@woodstock.com.br	369	2013−06−11 00:00:00	London	United Kingdo
14	10	53	Eduardo	Martins	eduardo@woodstock.com.br	43	2009−07−06 00:00:00	London	United Kingdo
15	10	53	Eduardo	Martins	eduardo@woodstock.com.br	54	2009−08−16 00:00:00	London	United Kingdo
16	10	53	Eduardo	Martins	eduardo@woodstock.com.br	109	2010−04−16 00:00:00	London	United Kingdo
17	10	53	Eduardo	Martins	eduardo@woodstock.com.br	238	2011−11−21 00:00:00	London	United Kingdo
18	10	53	Eduardo	Martins	eduardo@woodstock.com.br	261	2012−02−23 00:00:00	London	United Kingdo
19	10	53	Eduardo	Martins	eduardo@woodstock.com.br	283	2012−05−27 00:00:00	London	United Kingdo
20	10	53	Eduardo	Martins	eduardo@woodstock.com.br	335	2013−01−15 00:00:00	London	United Kingdo
21	43	52	Isabelle	Mercier	isabelle_mercier@apple.fr	11	2009−02−06 00:00:00	London	United Kingdo
22	43	52	Isabelle	Mercier	isabelle_mercier@apple.fr	140	2010−09−13 00:00:00	London	United Kingdo
23	43	52	Isabelle	Mercier	isabelle_mercier@apple.fr	163	2010−12−16 00:00:00	London	United Kingdo

Always clean-up!